

Module First:- Topic First:-Introduction and Approaches to Testing

Software Testing:-

Software Testing is the process of evaluating the software product, in order to assess the level of its quality. Moreover, this methodology is used to verify and validate the software product, against the functional and business requirements that were decided and specified prior to the software development process. It is one of the most significant phases of the [Software Development Life Cycle \(SDLC\)](#) that ensures delivery of best quality software to the end users & customers, in line with matching their expectations.

Before going through the mechanism of a process, it is vital for us to know its primary purpose. The main motive behind software testing is to attain maximum level of quality in a software product. The factor 'quality' is inversely proportional to the quantity of bugs or defects present in the software product. Higher the bugs, lower will be the quality and vice-versa.

Therefore, the goal of testing is to verify the quality of the software product by performing methodical elimination of bugs and defects. After all, testing is done with the vision of delivering the software product of maximum quality to the client or other stakeholders or customer in order to satisfy their needs and expectation.

Testing Objectives:-

Find defects – Identify bugs, errors, or inconsistencies before release.

Verify requirements – Ensure the system meets specified functional and non-functional requirements.

Validate user needs – Confirm the product works as expected for real users.

Assess quality – Evaluate reliability, performance, usability, security, and maintainability.

Reduce risk – Detect issues early to lower business, technical, or safety risks.

Ensure compliance – Check adherence to standards, regulations, or contractual obligations.

Build confidence – Provide stakeholders with confidence in the product's readiness.

Support decision-making – Supply data to decide whether to release, fix, or redesign.

fundamental testing principles, commonly taught in software testing / QA (

1. **Testing shows the presence of defects, not their absence**
Testing can find bugs, but it can never prove a system is completely defect-free.
2. **Exhaustive testing is impossible**
You cannot test all possible inputs and paths; testing must be selective and risk-based.
3. **Early testing saves time and money**
Finding defects early in the lifecycle reduces cost and effort to fix them.
4. **Defect clustering**
A small number of modules usually contain most of the defects (Pareto principle).
5. **Pesticide paradox**
Repeating the same tests will eventually stop finding new defects; test cases must be reviewed and updated.
6. **Testing is context dependent**
Different systems (banking, medical, games, web apps) require different testing approaches.
7. **Absence-of-errors fallacy**
Even if a system has few defects, it may still fail if it does not meet user needs.

Examples

Testing shows presence of defects

Example: You test a calculator and find it gives wrong results for $2 + 2$.

✓ You found a bug, but you can't say there are no other bugs.

Exhaustive testing is impossible

Example: A login page allows millions of username/password combinations.

You can't test all of them.

Early testing saves time and money

Example: Finding a mistake in requirements is cheaper than fixing it after the app is released.

Defect clustering

Example: Most bugs are found in the payment module, while other modules have very few bugs.

Pesticide paradox

Example: Running the same test cases every time finds no new bugs.

✓ You must add new test cases.

Testing is context dependent

Example: Testing a banking app needs security testing, but a game needs usability and performance testing.

Absence-of-errors fallacy

Example: An app has no bugs but is very slow and confusing for users

Examples

1. Testing shows presence of defects, not absence

Answer: Testing can find defects but cannot prove the software is error-free.

Example: Finding a wrong result in a calculator app.

2. Exhaustive testing is impossible

Answer: It is not possible to test all input combinations.

Example: A login page with many username and password combinations.

3. Early testing saves time and cost

Answer: Detecting defects early reduces fixing cost and effort.

Example: Catching requirement errors before coding starts.

4. Defect clustering

Answer: Most defects are found in a few critical modules.

Example: Payment module contains more bugs than other modules.

5. Pesticide paradox

Answer: Repeating the same tests does not find new defects.

Example: Old test cases fail to detect new bugs.

6. Testing is context dependent

Answer: Testing approach depends on the type of application.

Example: Banking apps need security testing, games need usability testing.

7. Absence-of-errors fallacy

Answer: Bug-free software may still fail if it does not meet user needs.

Example: An app works correctly but is too slow for users.

V-Model (Verification & Validation Model)

The V-Model is a software development and testing model where each development phase has a corresponding testing phase. Testing starts early and runs in parallel with developme

Simple Diagram (Exam-Oriented)

Requirements	←→	Acceptance Testing
System Design	←→	System Testing
Architecture Design	←→	Integration Testing
Module Design	←→	Unit Testing
Coding		

Explanation (Simple Words)

- Left side of V → **Development (Verification)**
- Right side of V → **Testing (Validation)**
- Bottom of V → **Coding**

Phases with Examples

1. **Requirements ↔ Acceptance Testing**
Example: Check if the software meets user needs.
2. **System Design ↔ System Testing**
Example: Test the whole system behavior.
3. **Architecture / High-Level Design ↔ Integration Testing**
Example: Test interaction between modules.
4. **Module / Low-Level Design ↔ Unit Testing**
Example: Test individual functions or modules.

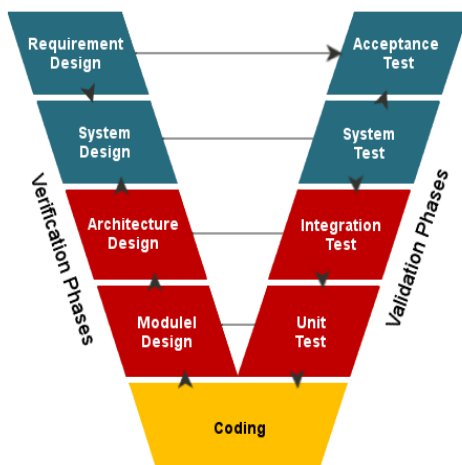
V model is an easy and simple approach for a successful software development and testing. Planning forms the baseline of V model. As there is a simultaneous test process along with the sequence of steps towards development, defect tracking becomes quite efficient. An ideal choice for small scale projects.

Advantages

- Easy to understand and manage
- Early defect detection
- Clear relationship between development and testing

Disadvantages

- Rigid model
- Not suitable when requirements change frequently
- No early working software



WHITE-BOX TESTING

White-box testing, sometimes called *glass-box testing*, is a test case design method

that uses the control structure of the procedural design to derive test cases. Using

White-box testing methods, the software engineer can derive test cases that

- (1) Guarantee that all independent paths within a module have been exercised at least once,
- (2) Exercise all logical decisions on their true and false sides,
- (3) Execute all loops at their boundaries and within their operational bounds, and
- (4) exercise internal data structures to ensure their validity.

- *Logic errors and incorrect assumptions are inversely proportional to the probability that a program path will be executed.* Errors tend to creep into our work when we design and implement function, conditions, or control that is out of the mainstream. Everyday processing tends to be well understood (and well scrutinized), while "special case" processing tends to fall into the cracks.

- *We often believe that a logical path is not likely to be executed when, in fact, it may be executed on a regular basis.* The logical flow of a program is sometimes counterintuitive, meaning that our unconscious assumptions about flow of control and data may lead us to make design errors that are uncovered only once path testing commences.

- *Typographical errors are random.* When a program is translated into programming language source code, it is likely that some typing errors will occur.

Many will be uncovered by syntax and type checking mechanisms, but others may go undetected until testing begins. It is as likely that a typo will exist on an obscure logical path as on a mainstream path.

Types:-

1. Statement Coverage

- Ensures that **every executable statement** in the code is tested at least once.
- Objective: Identify unused or unreachable code.

Example: Each line of code must run at least one time.

2. Branch Coverage (Decision Coverage)

- Ensures that **every decision point** (if/else, switch, loops) is executed for **both true and false conditions**.
- Objective: Validate all logical branches.

Example: if (x > 0)

3. Condition Coverage

- Tests **each individual condition** in a compound decision for both true and false.
- Objective: Validate logical expressions.

Example: `if (A && B)`

4. Multiple Condition Coverage

- Tests **all possible combinations** of conditions in a decision.
- Objective: Maximum logical validation.

Example For `if (A && B)`, combinations:

5. Path Coverage

- Ensures that **all possible execution paths** in the program are tested.
- Objective: Detect errors in complex logic flows.

Difficult to achieve for large programs due to many paths.

6. Loop Coverage

- Focuses on testing **loops** with different iterations.
- Tests:
 - Zero iterations
 - One iteration
 - Multiple iterations
 - Maximum limit

7. Control Flow Testing

- Analyzes the **flow of control** through the program using a control flow graph.
- Objective: Verify sequence and decision logic.

8. Data Flow Testing

- Focuses on **variable usage** (definition and use of variables).
- Objective: Detect issues like uninitialized variables or incorrect data usage.

9. Mutation Testing

- Small changes (*mutations*) are made to the source code to check if test cases can detect errors.
- Objective: Measure the effectiveness of test cases.

Summary Table

Type	Focus Area
Statement Coverage	Code statements
Branch Coverage	Decision points
Condition Coverage	Boolean conditions
Path Coverage	Execution paths
Loop Coverage	Loop behavior
Data Flow Testing	Variable usage
Mutation Testing	Test strength

BLACK-BOX TESTING

Black-box testing, also called behavioral testing, focuses on the functional requirements of the software. That is, black-box testing enables the software engineer to derive sets of input conditions that will fully exercise all functional requirements for a program. Black-box testing is not an alternative to white-box techniques. Rather, it is a complementary approach that is likely to uncover a different class of errors than white-box methods.

Black-box testing attempts to find errors in the following categories:

- (1) incorrect or missing functions,
- (2) interface errors,
- (3) errors in data structures or external data base access,
- (4) behavior or performance errors, and
- (5) initialization and termination errors.

Tests are designed to answer the following questions:

- How is functional validity tested?
- How is system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation

Types of Black Box Testing

1. Functional Testing

- Verifies that each function of the software works as per requirements.
- Focuses on **what the system does**, not how.

Example: Login, registration, payment processing.

2. Non-Functional Testing

- Tests system attributes like performance, usability, and reliability.

Includes: **Performance Testing** , **Load Testing** ,**Stress Testing** ,**Security Testing** **Usability Testing** ,**Compatibility Testing**

3. Regression Testing Ensures that new changes do not break existing functionality.

4. Smoke Testing Basic tests to check whether the build is stable for further testing.

5. Sanity Testing Quick testing after minor changes to verify specific functionality.

6. Acceptance Testing Confirms the system meets business requirements.

Types:

- **User Acceptance Testing (UAT)**, **Alpha Testing**, **Beta Testing**

7. Equivalence Partitioning Divides input data into valid and invalid partitions.

Example:

Age field (1–100):

Valid → 18–60

Invalid → <18, >60

8. Boundary Value Analysis (BVA) Tests values at the **edges** of input ranges.

Example:

Values tested: 17, 18, 60, 61

9. Decision Table Testing Tests combinations of conditions and corresponding actions.

10. State Transition Testing Tests behavior when the system changes states.

Example: ATM machine states.

11. Error Guessing Tester uses experience to guess possible error-prone areas.